Computer Architecture and System Software

Lecture 08: Memory Hierarchy

Instructor:
David Bosc
Applied Computer Science
University of Winnipeg

Announcements

- Important Dates:
  - Last day to voluntary withdrawal is this Friday, March 13th (This Friday)
  - Assignment 3 Due March 13th
- Supplementary Materials Uploaded to Nexus
  - Not mandatory to utilize by any means
    - Textbook Video Lectures YouTube Playlist
    - Game Programming Patterns: Data Locality

Chapter 6

The Memory Hierarchy

6.1 – 6.3

The Memory Hierarchy

- To this point:
  - Simple model of a computer system
    - CPU that executes instructions
    - Memory system that holds instructions and data for CPU
      - Linear array of bytes
      - CPU can access each memory location in a constant amount of time
- Effective model, but does not reflect reality

The Memory Hierarchy

- In practice, a memory system is a hierarchy of storage devices
  - Different capacities, costs, and access times
- For example:
  - CPU registers: hold the most frequently used data
  - Cache memories: act as a staging area for a subset of data & instructions in (relatively) slow main memory
  - Main memory: act as staging area for data stored on large, slow disks
  - HD: May act as a staging area for data stored on tapes or other machines connected by networks

The Memory Hierarchy

- Why does this work?
  - Well-written programs tend to access the storage at any particular level more frequently than the next lower level
    - Thus: storage at next level can be slower
      - Means it can be larger and cheaper per bit
- Overall effect:
  - A large pool of memory that is as cheap as memory at the bottom of hierarchy
  - Serves data to programs at the rate of the fast storage near the top

The Memory Hierarchy

- Why do you need to know about the hierarchy?
  - It has a big impact on the performance of your applications
  - e.g.
    - Data in a CPU register can be accessed in 0 cycles
    - Cache: 1 – 10 cycles
    - Main memory: 50 to 100 cycles
    - Disk: 20, 000, 000 cycles

Recall: Clock Cycle

- The frequency at which a computer can execute instructions
- Controlled by a crystal that vibrates when electric current is applied
- Important for ordered timing of instructions
- Modern processors are on the order of GHz (billions of instructions per second)

The Memory Hierarchy

- If you understand this hierarchy:
  - You can write application programs so that their data is stored higher in the hierarchy when the CPU needs them
- How?
  - Locality: programs with good locality tend to access the same set of data items over and over again
    - Or they tend to access sets of nearby data items
    - Tend to access more data items from the upper levels of the hierarchy than programs with poor locality

The Memory Hierarchy

6.1 Storage Technologies

Storage Technologies

- Success of computer technology stems from progress in storage technology
- Early computers had a few kilobytes of random-access memory
  - Earliest IBM PCs did not have a hard disk
  - Changed with IBM PC-XT (1982) which had 10 MB disk
  - By 2000 typical machines had 1024 times as much disk storage
    - Ratio was increasing by a factor of 10 every two or three years

RAM and ROM

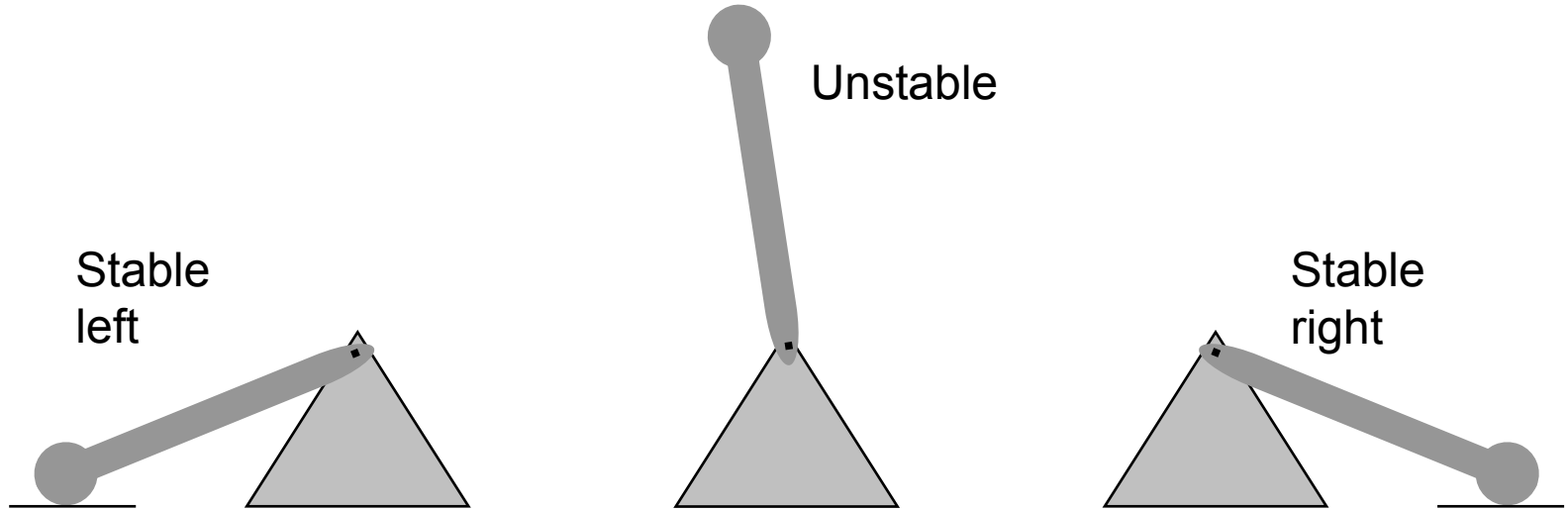
- Two types of storage: Random Access Memory (RAM) and Read Only Memory (ROM)
  - Names are historical and do not reflect true behaviour of the memory anymore
- Read only
  - Originally used mainly for firmware, CDs, hard-wired circuits etc.
- Random Access
  - Called ‘random’ to differentiate from magnetic tapes that had to be accessed sequentially
  - Should really be called ‘non-sequential’

Random Access Memory (RAM)

- Stores data in the computer while it is on
- Implements a simple interface
  - Looks like a sequence of cells, each storing a fixed number of bits
  - Controlled by three sets of lines (wires)
- Address lines: specifies the address in which to read/write
- Data lines: used to transmit data to/from memory cells
- Control line: Indicates if operation is read or write

Static RAM (SRAM)

- Static RAM stores each bit in bistable memory
  - Implemented with a six-transistor circuit
- Can stay indefinitely in either of two voltage configurations (states)
  - Any other state is unstable
    - Circuit will quickly move toward one of the stable states



Static RAM

- Due to its bistable nature, an SRAM memory cell will retain its value indefinitely
  - As long as it is kept powered
- Even with a disturbance, the circuit will return to the stable value
  - *i*.e. once the noise is removed

Dynamic RAM

- Stores each bit as a charge on a capacitor
- DRAM storage can be made very dense
  - Each cell consists of a capacitor and a single access transistor
- DRAM is very sensitive to disturbance
- Capacitor voltage will never recover from a disturbance
- Also, capacitor will lose it charge around 10 to 100 milliseconds
  - Due to various sources of leakage

Dynamic RAM

- Fortunately: CPU cycles are typically measured in nanoseconds
  - CPU point of view: Retention time is quite long
- Due to leakage, the memory must be periodically refreshed
  - Entails reading every bit & then rewriting it
  - Error correcting codes are also used to correct bits that have been misread/corrupted

SRAM vs DRAM

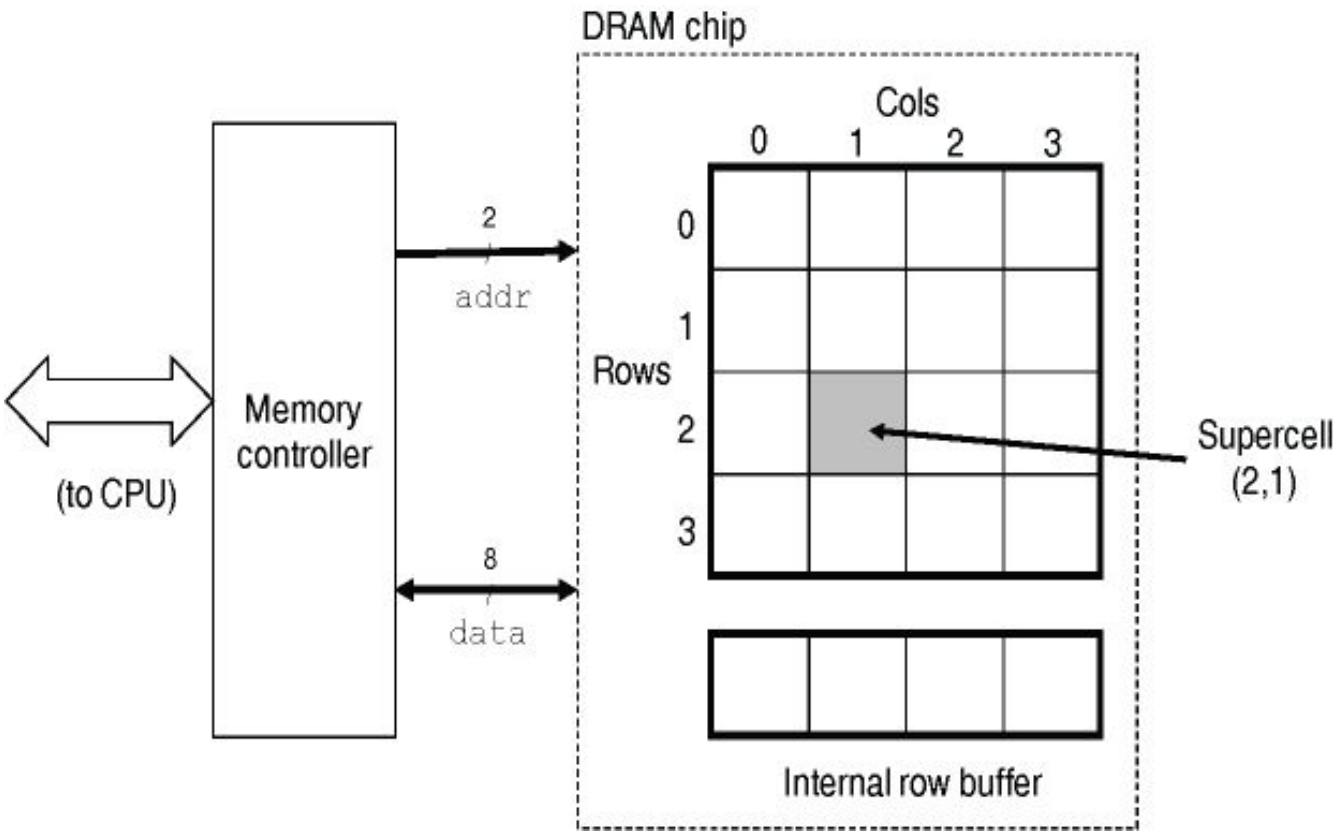
- Static RAM (SRAM)
  - Each bit is stored in bistable memory
    - Memory will store values unless disturbed
    - 1 bit = 6 transistors
    - Fast and expensive
- Dynamic RAM (DRAM)
  - Stores each bit as a charge on a capacitor
  - Has to be refreshed on regular basis
  - Uses 1 transistor per bit
  - Can be made very dense (lots of bits per inch)
  - 100X cheaper
  - 10X slower

Comparing SRAM and DRAM

	Transistors per bit	Relative access time	Relative cost	Applications
SRAM	6	1X	100X	Cache memory
DRAM	1	10X	1X	Main/video memory

Conventional DRAMs

High level view of a 128-bit (16 B) 16x8 DRAM chip



Conventional DRAMs

- Cells (bits) in a DRAM chip are partitioned into d supercells
  - Each consists of w DRAM cells (bits)
  - A d x w DRAM stores a total of dw bits of information
  - Supercells are organized as a rectangular array
    - r rows & c columns
    - $rc = d$
  - Each supercell has an address of the form (i,j)
    - i denotes the row
    - j denotes the column

Conventional DRAMs

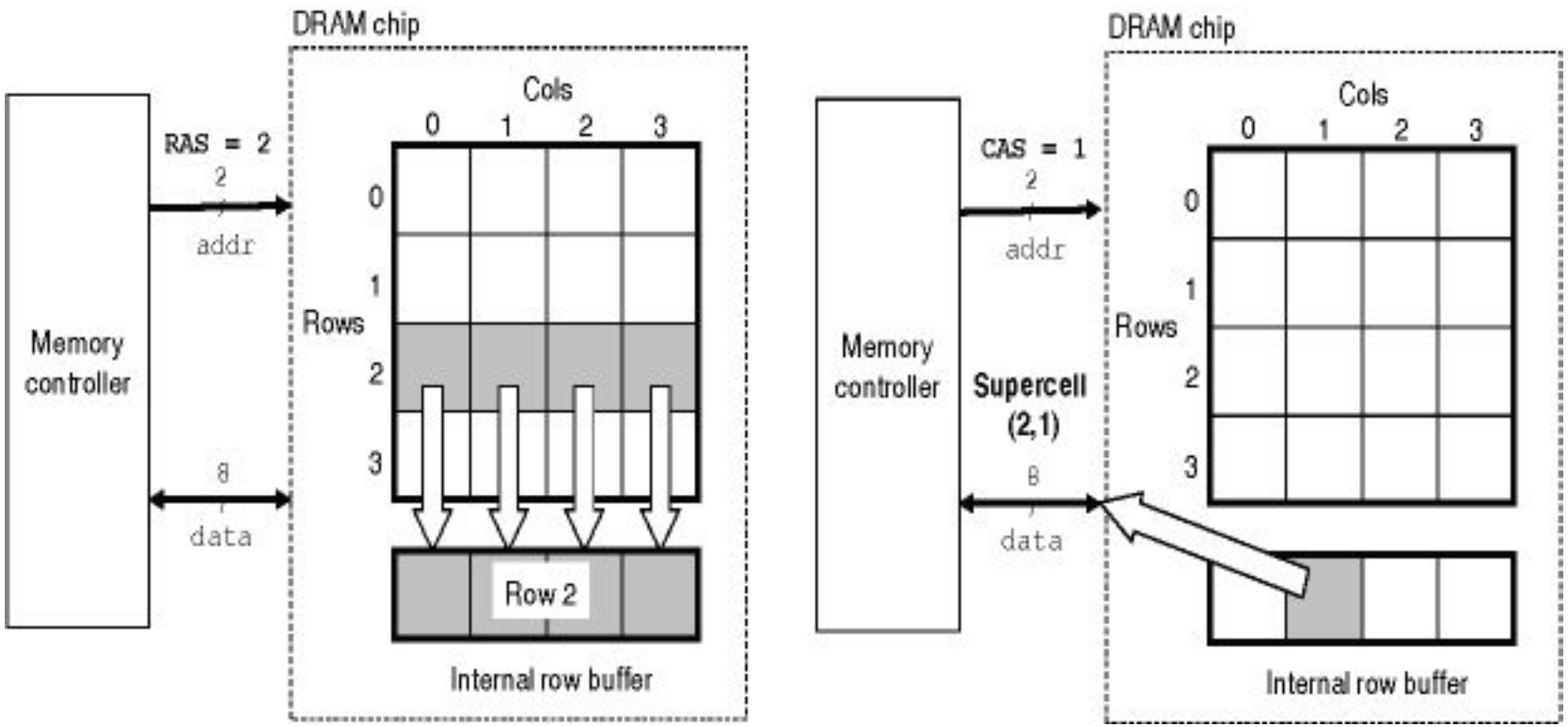
Example

- 128-bit (16B) 16 x 8 DRAM chip
  - d supercells of w bits each
  - 16 supercells of 8 bits each
  - In this case, supercell configuration is 4 x 4

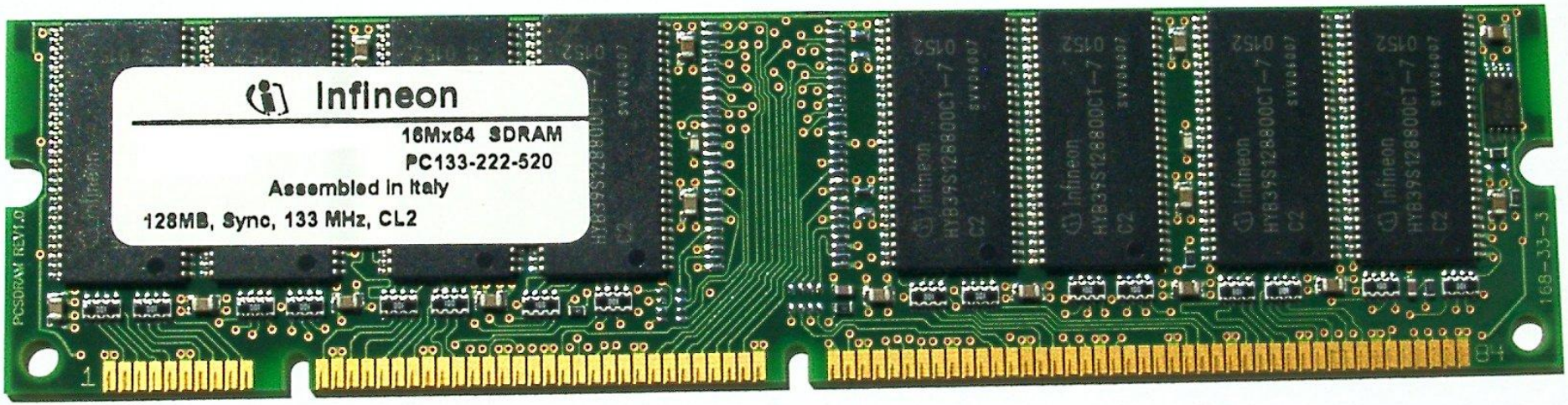
Conventional DRAM

- Information flows in and out of the chip via external connectors called pins
- Each pin carries a 1-bit signal
- Example shows 8 data pins that can transfer 1 byte in/out of the chip
- Also shows 2 address pins that identify row/column of supercell
- Other control pins are not shown

Conventional DRAMs



SDRAM Module



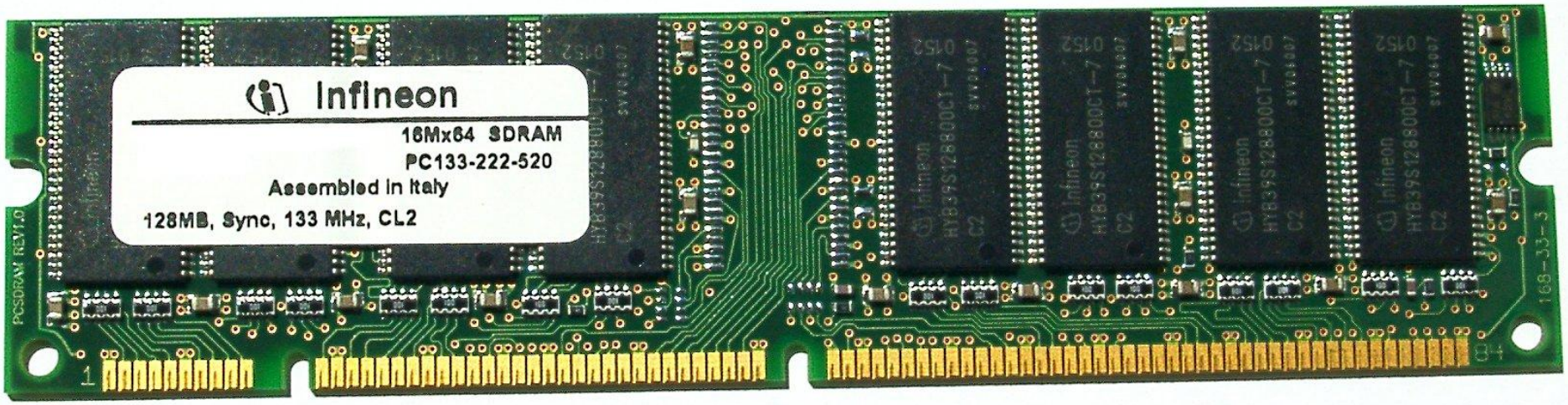
Conventional DRAMs

- Each DRAM chip is connected to a memory controller
- Control can transfer w bits at a time to/from chip
- Data is read by sending row address first
  - The whole row is copied to internal row buffer
- Then, column address is sent and w bits are retrieved
- Chips are designed this way to reduce the number of address pins on the chip

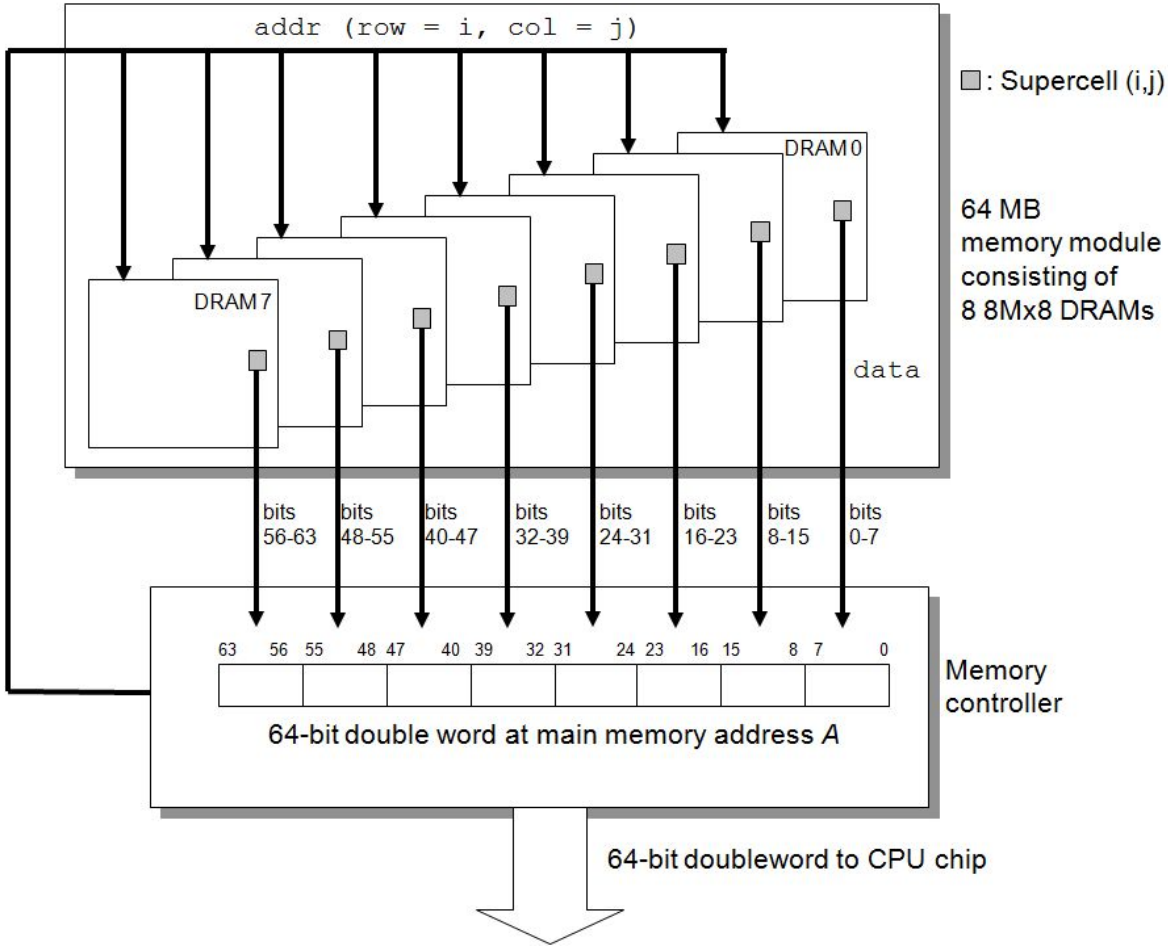
Memory Module

- DRAM chips are packaged in memory modules that plug into expansion slots on the motherboard
- Common packages include:
  - 168-pin Dual Inline Memory Module (DIMM)
    - Transfers data to and from memory controller in 64-bit chunks
  - 72-pin Single Inline Memory Modules (SIMM)
    - Transfer data in 32-bit chunks

SDRAM Module



Memory Module



Memory Module

Example

- Module stores a total of 64 MB
  - Uses eight 64-Mbit 8M x 8 DRAM chips
    - Numbered 0-7
- Each supercell consists of one byte of main memory
- Each 64-bit quadword at address A in main memory is represented by 8 supercells
  - *i*.e. the supercells whose corresponding address is (i,j)

Memory Module

- To retrieve a 64-bit quadword at address A
  - Memory controller converts A to a supercell address (i,j)
  - Sends address to memory module
  - Memory module broadcasts i and j to each DRAM
  - DRAM outputs 8-bit contents of its (i,j) supercell
  - Module collects these outputs and forms them into a 64-bit quadword
  - Quadword returned to memory controller

Memory Module

- Main memory is an aggregate of multiple memory modules
  - Each memory module stores part of the address space
  - A module consists of DRAM chips
  - DRAM chips consist of supercells containing a number of bits
- 4 GB of memory example:
  - 1024 Mb (128 MB) DRAM chip
  - 8 chips on a module (128MB x 8 chips = 1024 MB = 1GB)
  - 1GB Requires 30 address lines
  - Only ¼ of address spaces
  - Need 4 banks of modules (32 DRAM chips) to get 4GB
  - Memory controller determines which module to use based on last 2 lines

Enhanced DRAMs

- There are many kinds of DRAM memories
  - New kinds appear regularly
    - To keep up with processor speeds
  - Each is based on the conventional DRAM cell
    - With optimizations that improve the speed with which the basic DRAM cells can be accessed

Enhanced DRAMs

- Fast Page Mode DRAM (FPM DRAM)
- Extended data out DRAM (EDO DRAM)
- Synchronous DRAM (SDRAM)
- Double Data-Rate Synchronous DRAM (DDR SDRAM)
  - DDR2
  - DDR3
- Rambus DRAM (RDRAM)
- Video RAM (VRAM)

Enhanced DRAMs

- Fast Page Mode DRAM (FPM DRAM)
  - Keeps a row in the internal buffer
  - Executes successive read/writes on the row
  - Advantage: Save time spent on transmitting the same row address on successive read/writes

Enhanced DRAMs

- Extended data out DRAM (EDO DRAM)
  - Similar to FPM RAM, but with slightly different signal timings
  - Results in small efficiency boost

Enhanced DRAMs

- Synchronous DRAM (SDRAM)
  - Synchronous refers to the timing of signals with the computer clock cycle
  - Most significant change is introduction of internal 'address banks'
  - This allows a second memory read to begin while the first memory read is in progress
  - Keeps data bus continuously busy (more efficient)

Enhanced DRAMs

- Double Data-Rate Synchronous DRAM (DDR SDRAM)
  - Access time is the same as SDRAM
  - Transfer of data is doubled for DDR SDRAM
  - DDR2, DDR3 have same average transfer rate, but they transfer more data at one time
  - This results in less memory reads on average
  - Processor doesn't have to wait on memory as much since it can work on bigger chunks of information at one time

Enhanced DRAMs

- Both RDRAM and VRAM are obsolete
- Rambus DRAM (RDRAM)
  - Similar to SDRAM
  - Performed marginally better but had significantly higher heat output and cost
- Video RAM (VRAM)
  - Has extra set of data output pins that are read-only
  - These are optimized for high throughput to a graphics card

Aside: Graphics Processing

- Optimizations that began with VRAM and continued with more modern graphics card memory are seeing increasing use in scientific computing

- Idea:
  - Graphic Processing Units (GPUs) started by optimizing calculations in parallel for display on monitor
  - Scientific community began utilizing this strategy for time consuming mathematical problems

Nonvolatile Memory

- DRAM and SRAM are volatile since they lose their information if the power supply is turned off
- Nonvolatile memory stores data even when powered off
- Called Read Only Memory (even though some can be written to as well)
- ROMs are distinguished by the number of times they can be reprogrammed (written to)

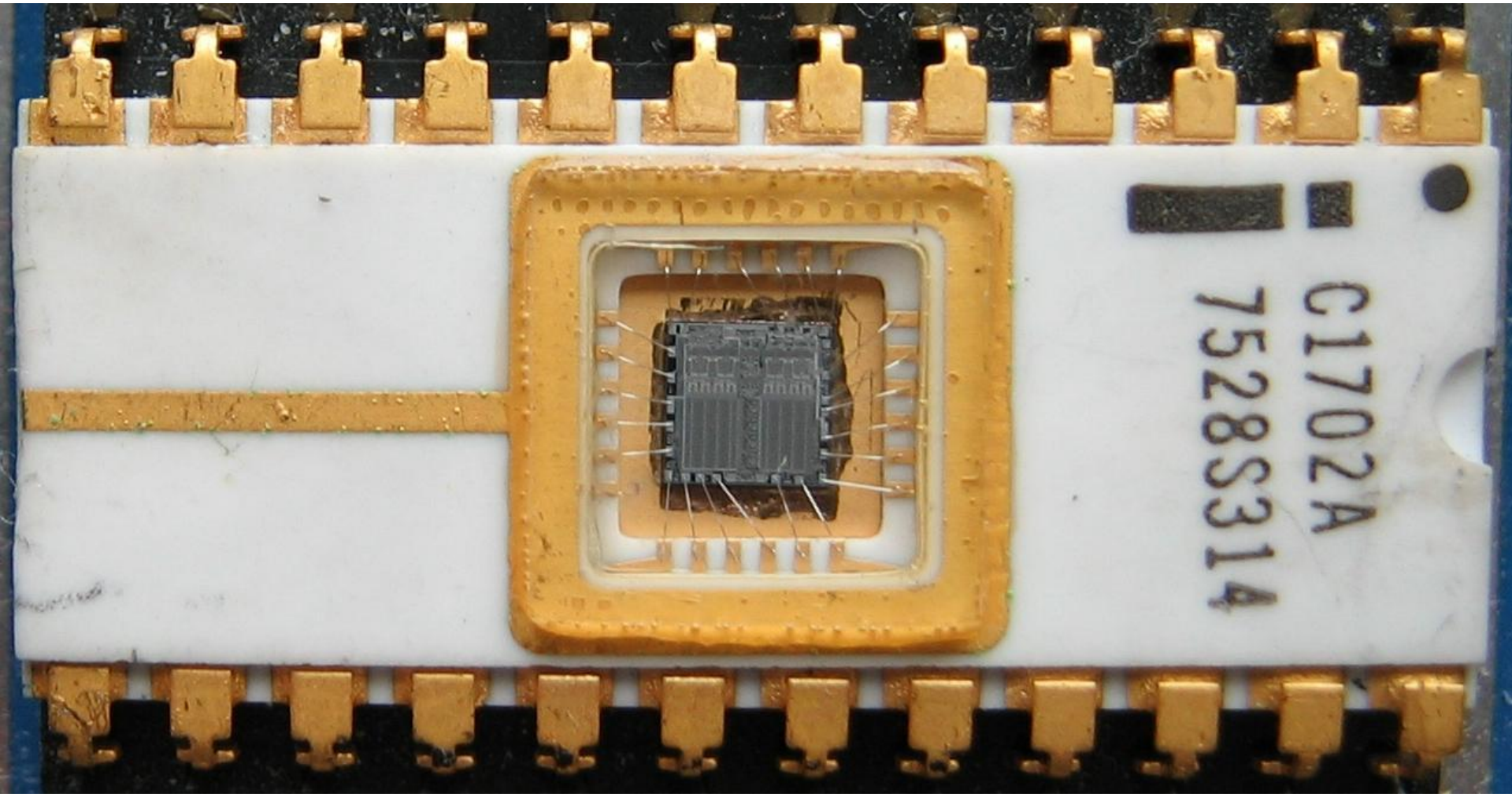
Nonvolatile Memory

- Programmable ROM (PROM): can be programmed once
  - By blowing a fuse
- Erasable Programmable PROM (EPROM):
  - Has a transparent quartz window
  - Cells are cleared to zeros by shinning ultraviolet light
  - Programmed with special device
  - Can be programmed ~1000 times
- Electrically Erasable PROM (EEPROM):
  - Can be reprogrammed in place
  - Doesn't need special device to program like EPROM
    - Example: flash memory
  - Can be reprogrammed ~10^5 times

Nonvolatile Memory

- PROM applications:
  - Programs that run on PROM are called firmware
  - First thing to run when power is turned on
  - Remote controls, video game consoles, appliances, etc
- EPROM applications:
  - Basic input output system (BIOS) (also considered firmware)
  - Bootstrap OS
- EEPROM applications:
  - USB, RAM, solid state drives, etc.

EPROM



Accessing Main Memory

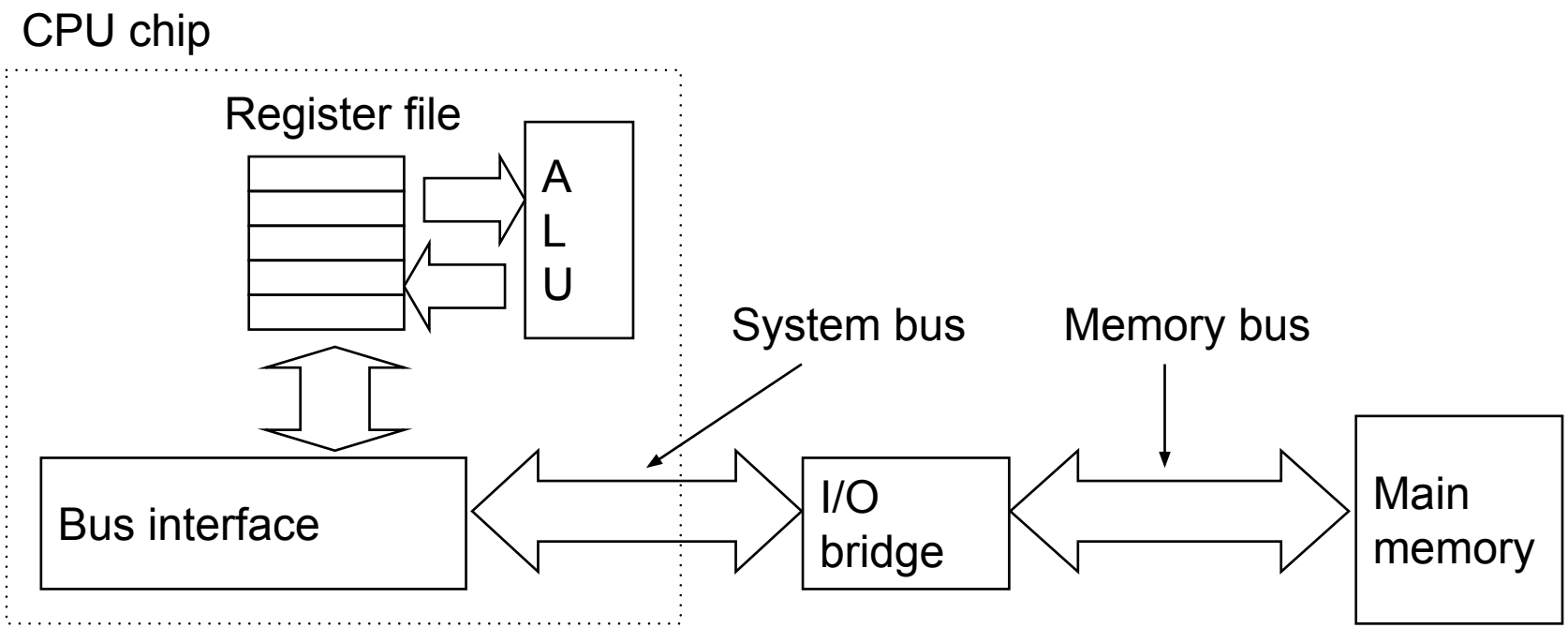
- Data flows between the processor and the DRAM main memory over shared electrical conduits
  - Called buses
- Each transfer is accomplished with a series of steps
  - Called a bus transaction
  - Read transaction: data from main memory
  - Write transaction: data to main memory

Accessing Main Memory

- Bus is a collection of parallel wires that carry address, data, and control signals
  - Data and address signals can share the same set of wires or use different sets
  - Also, more than two devices can share the same bus
- Control wires carry signals that:
  - Synchronize transaction
  - Identify the type of transaction
    - Destination of transaction
    - Read or write
    - Information on bus address or data

Accessing Main Memory

- Configuration of typical desktop system



Components of a Memory System

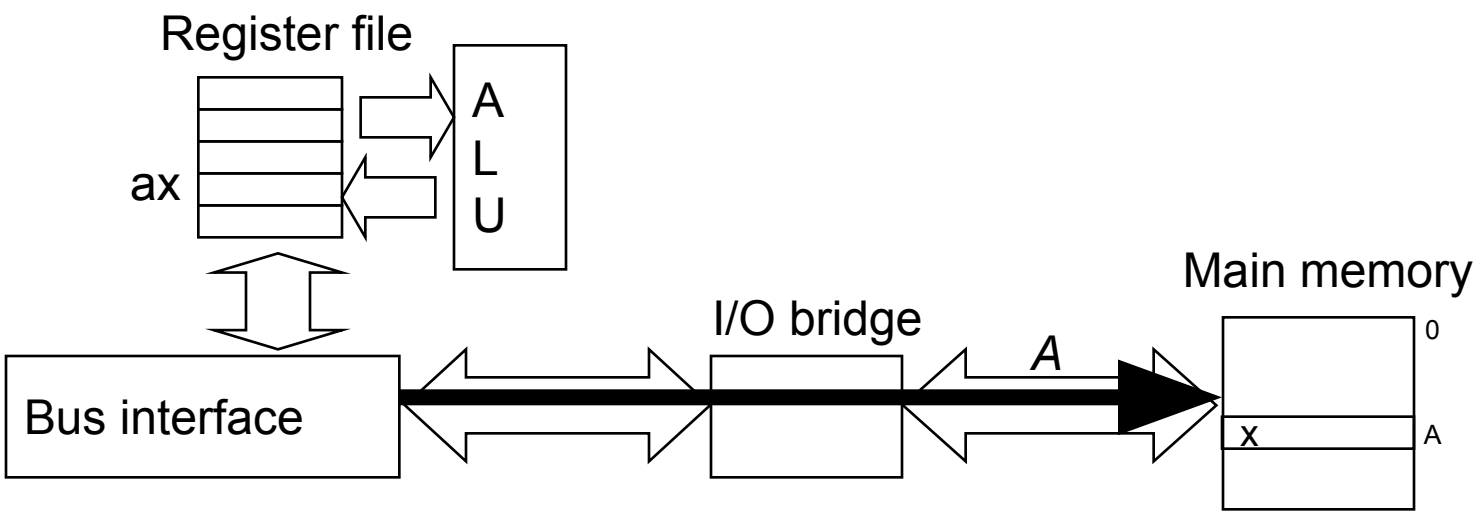
- Bus Interface: Controls the system bus
- System Bus:
  - Main connection between CPU and computer
  - 32 or 64 data lines (wires)
  - Small number of control lines
- I/O Bridge:
  - Bridge to memory bus and I/O bus
  - Directs CPU requests for memory or I/O
  - Also connects to I/O bus what is shared by I/O devices
- Memory Bus: main connection between I/O bridge and memory controller

Components of a Memory System

- Memory Controller
  - Manages memory chips
  - Executes store/load requests
- Memory Chips: store actual data
  - Two types of operations:
    - Read transaction
    - Write transaction

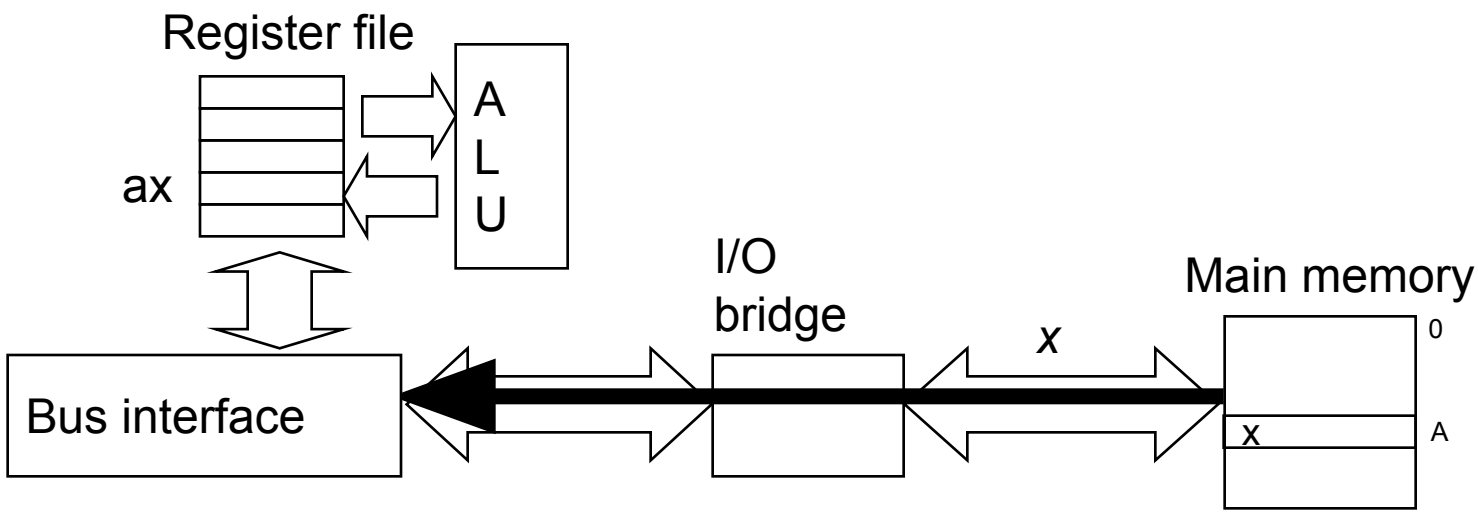
Read Transaction: mov ax, A

- Bus interface initiates a read transaction on the bus
- CPU places address A on the system bus via bus interface
- I/O bridge interprets request as memory read
- Forwards it on to memory bus



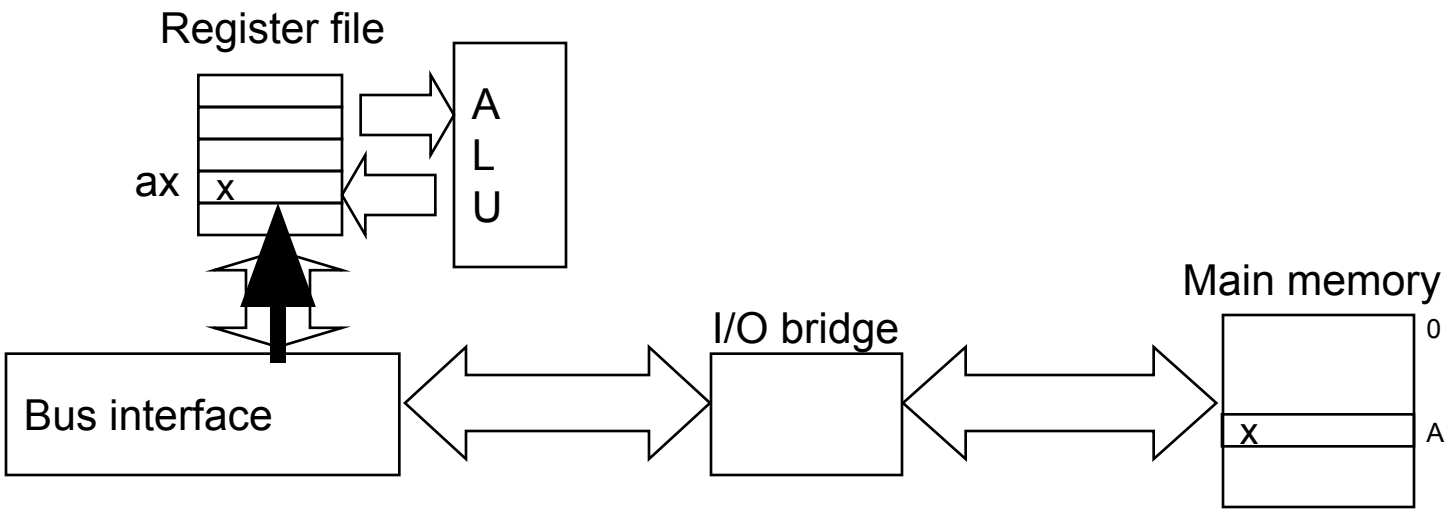
Read Transaction: mov ax, A

- Main memory controller
  - Senses the address signal on memory bus and reads it
  - Fetches the data word from DRAM and writes it to memory bus
  - I/O bridge translates the memory bus signal into a system bus signal and passes it along to the system bus



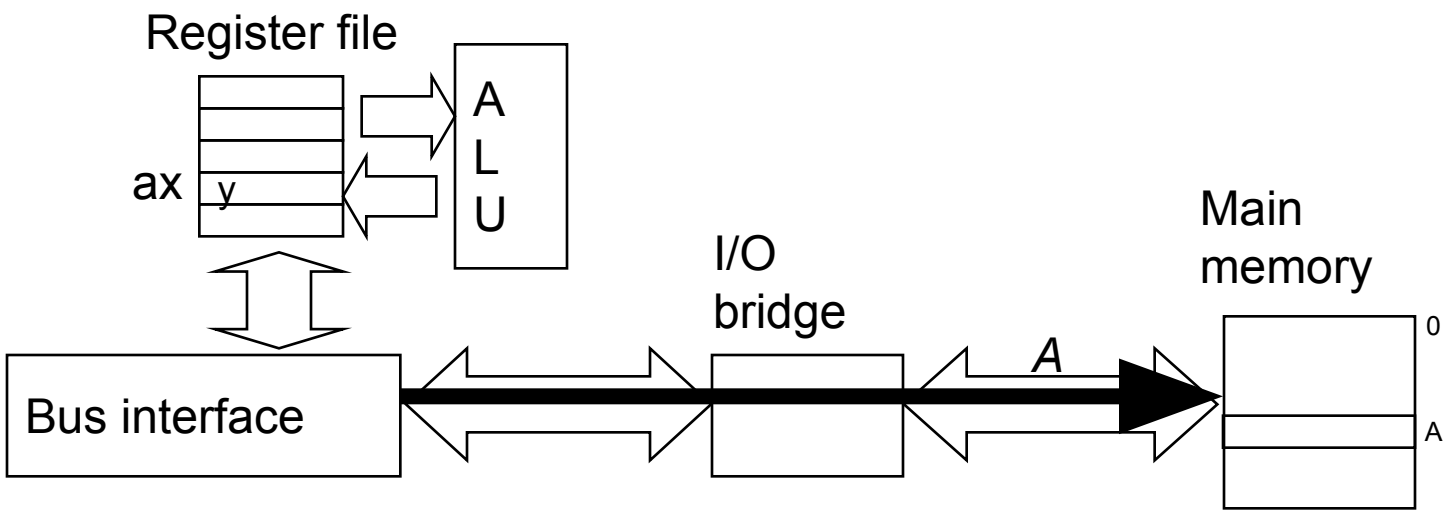
Read Transaction: mov ax, A

- CPU:
  - Senses data on system bus
  - Reads it from the bus
  - Copies it to register ax



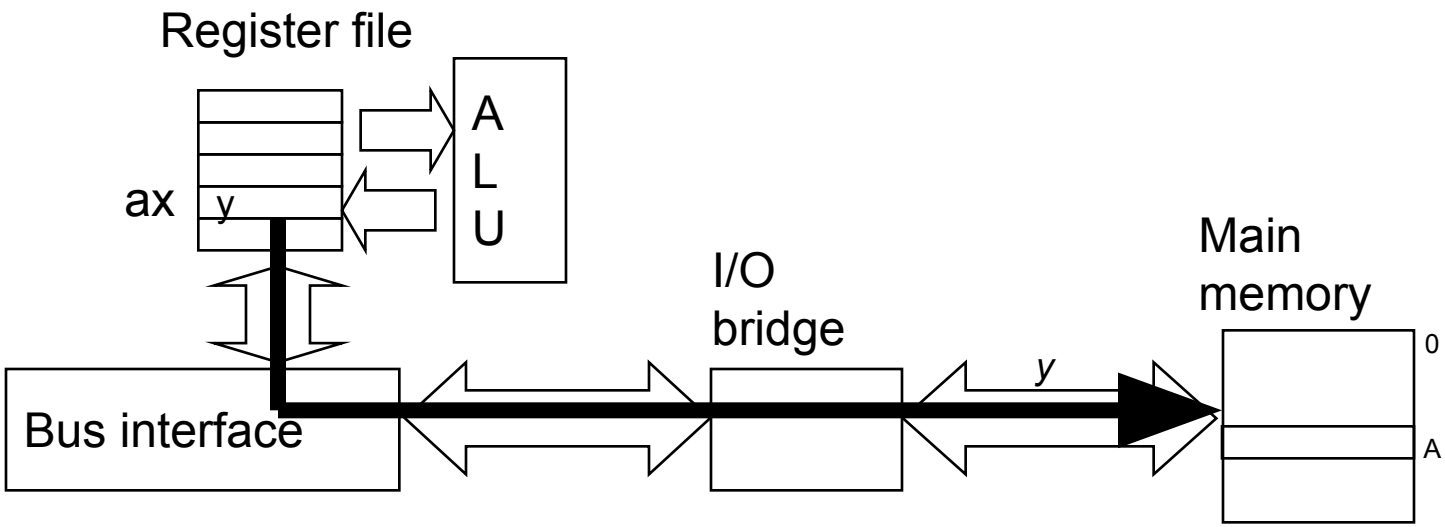
Write Transaction: mov A, ax

- CPU places the address on the system bus
- I/O bridge interprets requests as memory write
- Forwards it on memory bus
- Memory reads address from memory bus and waits for data



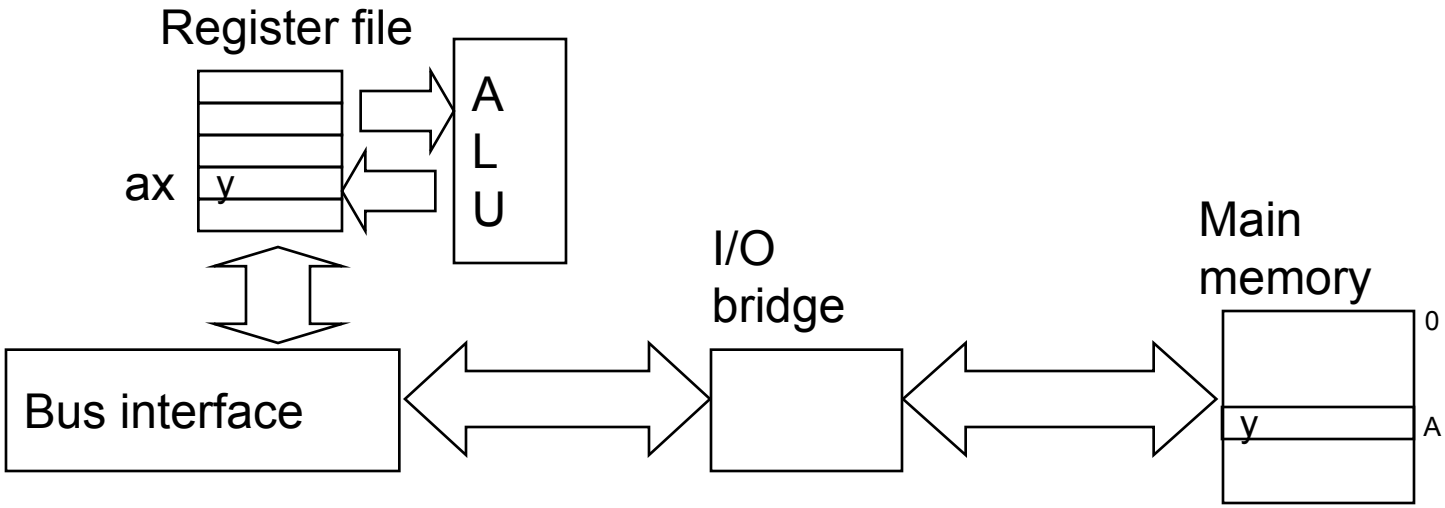
Write Transaction: mov A, ax

- CPU copies the data word in ax to system bus
- I/O bridge copies data to memory bus



Write Transaction: mov A, ax

- Main memory reads the data word from the memory bus and stores bits in memory



Limitations of Main Memory

- Size: Main memory is too small to hold all our data
- Volatility: Main memory does not store data when computer is off
- Solution:
  - Disk storage is
    - Large
    - Cheap
    - Non-volatile

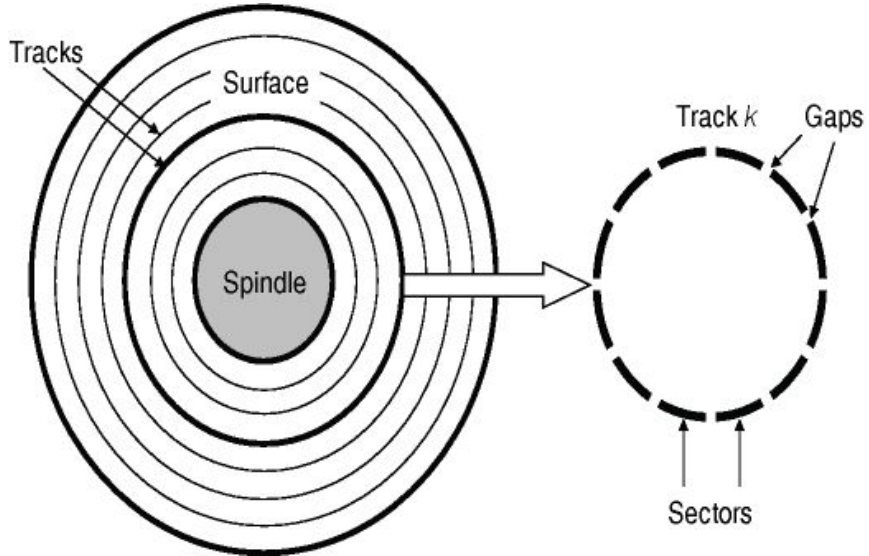
Hard Disks

Disk Geometry

- Disks are constructed from platters
  - Each platter consists of two sides or surfaces
    - Each side coated with a magnetic recording material
- A rotating spindle in the center spins the platter at a fixed rotation rate
  - Typically between 5400 and 15000 revolutions per minute (RPM)
- A disk typically contains one or more platters
  - Encased in a sealed container
  - Why?

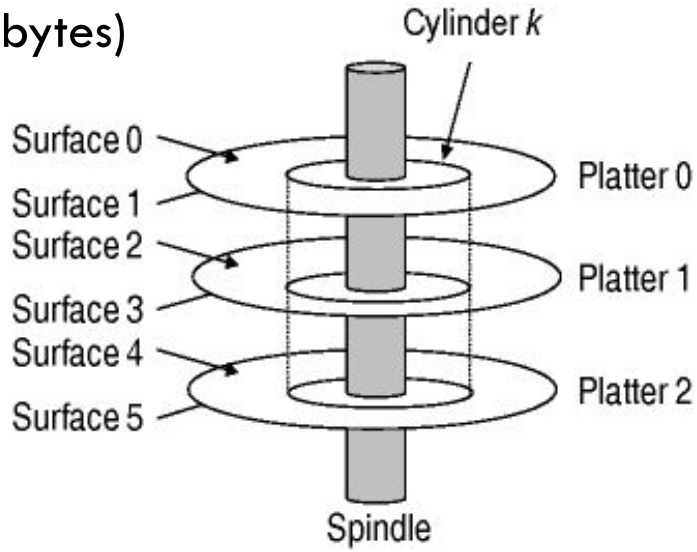
Disk Geometry

- Platter
  - Thin disks coated with magnetic recording material
  - Placed on a rotating spindle in the center of the platter
  - Spin at 5400 to 15000 RPM
  - Has two surfaces (*i.e.* both sides store data)
- Surface comprises a collection of concentric rings called tracks



Disk Geometry

- Track:
  - Partitioned into a collection of sectors
- Sector
  - Contains an equal number of bits (typically 512 bytes)
  - Separated by gaps where no data is recorded
    - Gaps store formatting bits that identify sectors
- Cylinder
  - A collection of tracks
  - Located in the same location on each surface
  - # of tracks per cylinder = # of surfaces
- Numbering
  - Surfaces, tracks (cylinders), and sectors are numbered
- Location is defined as (surface, cylinder, sector)



Disk Capacity

- Maximum # of bits that can be recorded on the HD depends on
  - Recording density (bits/in): # of bits in 1-inch of surface segment of a track
  - Track density (tracks/in) : # of tracks in a 1-inch segment of the radius extending from the center of the patter
  - Areal density (bits/in^2): The product of the recording density and the track density

$$\text{Disk Capacity} = \frac{\#\text{ bytes}}{\text{sector}} \times \frac{\text{average }\#\text{ sectors}}{\text{track}} \times \frac{\#\text{ tracks}}{\text{surface}} \times \frac{\#\text{ surfaces}}{\text{platter}} \times \frac{\#\text{ platters}}{\text{disk}}$$

Disk Capacity

Example

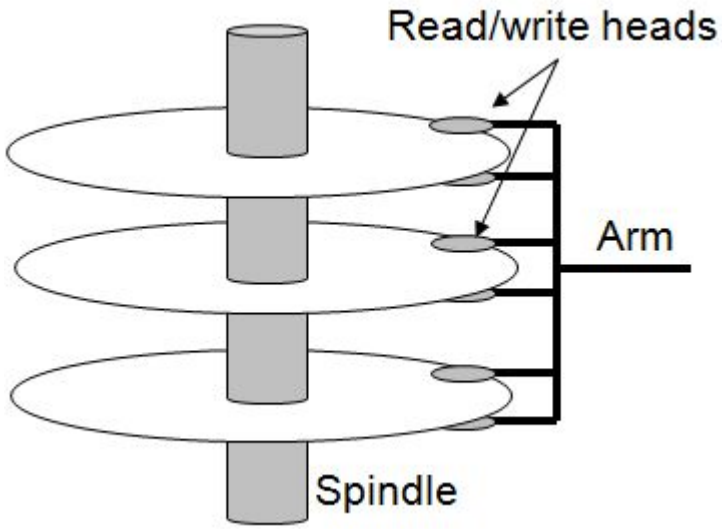
- 5 platters
- 512 bytes per sector
- 20,000 tracks per surface
- Average of 300 sectors per track

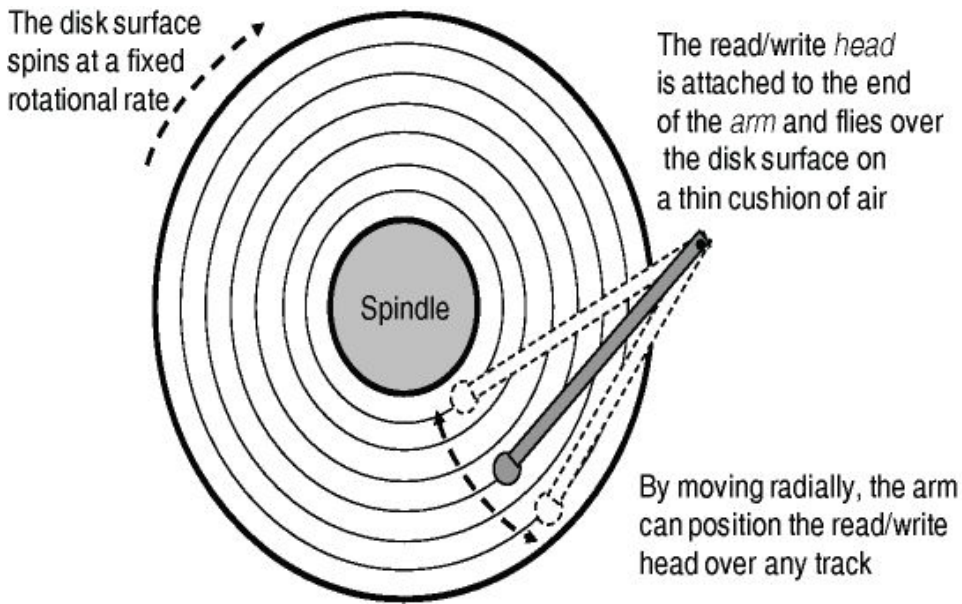
Disk Capacity

- Long ago all tracks had same # of sectors
  - Easy to implement
  - Efficient for low density recording
  - **Problem:** Inefficient for high density recording (too many wasted bits)
- Solution:
  - Set of tracks is partitioned into zones
  - Each zone has a fixed # of sectors per track
  - Disk controller keeps tracks of zones

Disk Operations

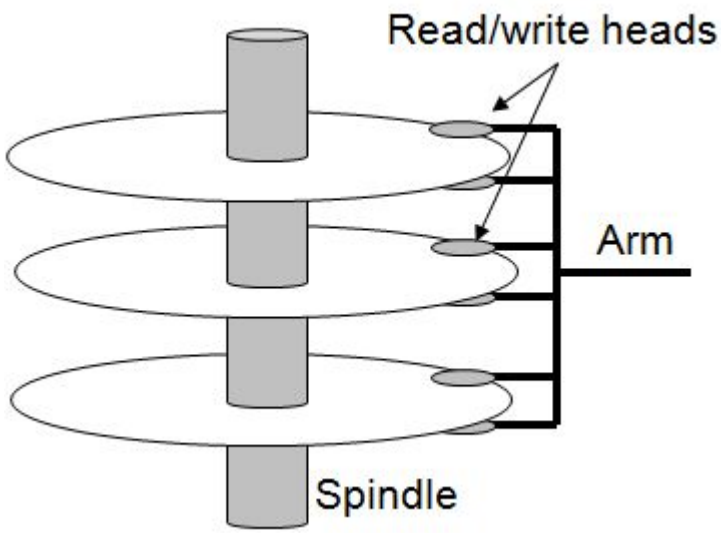
- Magnetic material on surface stores bits
- Written and read by passing over area of bit with a r/w head
- r/w head attached to actuator arm
- Actuator arm can position head any where on radial axis of disk

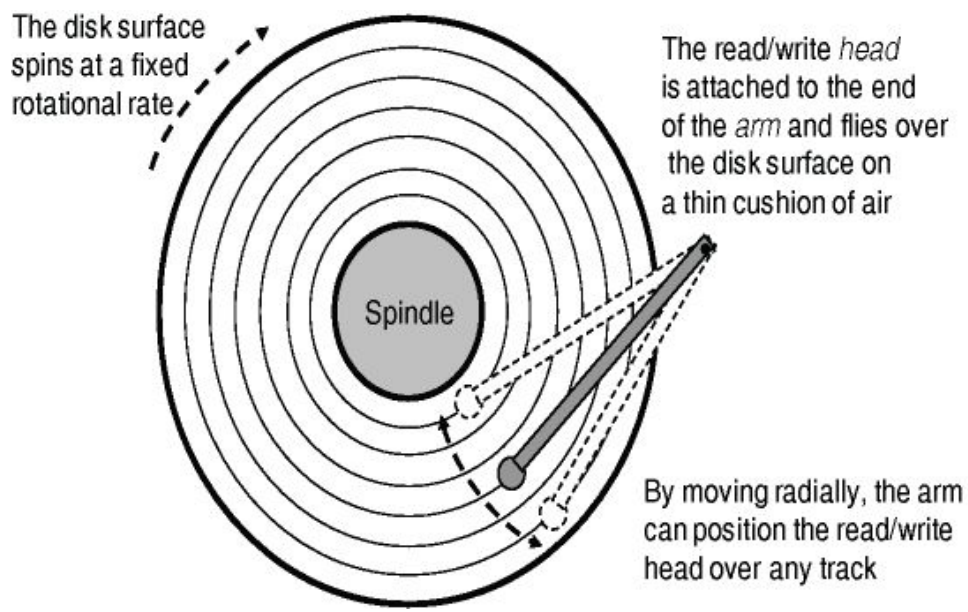




Disk Operations

- Once head is positioned, rotation of disk will bring desired sector under head
- Disk spins so fast, head flies 0.1 micron above surface at 80 km/hr
- Disk read and write data in sector-sized blocks





Disk Operations

- To perform r/w operation:
  - Seek: move head to correct cylinder
  - Wait for sector: rotation of disk will bring sector under the head
  - R/W sector:
    - Read: send bits from head to controller as sector passes head
    - Write: send bits from controller to head as sector passes head

Access Time

- How long to read or write a sector?
  - Seek time: time to move head to correct cylinder
    - Depends on previous position of head
    - Speed of actuator arm
    - Average seek time: measured by averaging time of several thousand seeks
    - Max seek time can be as high as 20ms
  - Rotational latency: time to wait before sector passes head
    - Depends on rotation speed of disk
    - Location of sector at time of operation
    - Worst case: head just missed sector and must wait for complete rotation
    - Average case: worst case divided by 2
    - Max. rotational latency in seconds (1/RPM) x 60 seconds

Access Time

- Transfer time (throughput): amount of time to r/w a sector
  - Depends on rotation speed of disk & # of sectors per track
  - Approx = (1/RPM) x (1/avg. # sectors per track) x 60 seconds

- Estimate the avg. time to access the contents of a disk sector as the sum of the avg. seek time, avg. rotational latency, and avg. transfer time
  - Example:
    - Disk 7200 RPM, 9ms seek, 400 sectors/track
    - Access time = 13.19 ms

Access Time Observations

- Seek time and rot. lat. dominate access time
- 2x seek time is good estimate of access time
- Access time to read 512-byte sized block:
  - SRAM = 256 ns
  - DRAM = 4000 ns
  - Disk = 10 ms
- Disk access time is roughly 40000X greater than SRAM, and 2500X greater than DRAM

Logical Disk Blocks

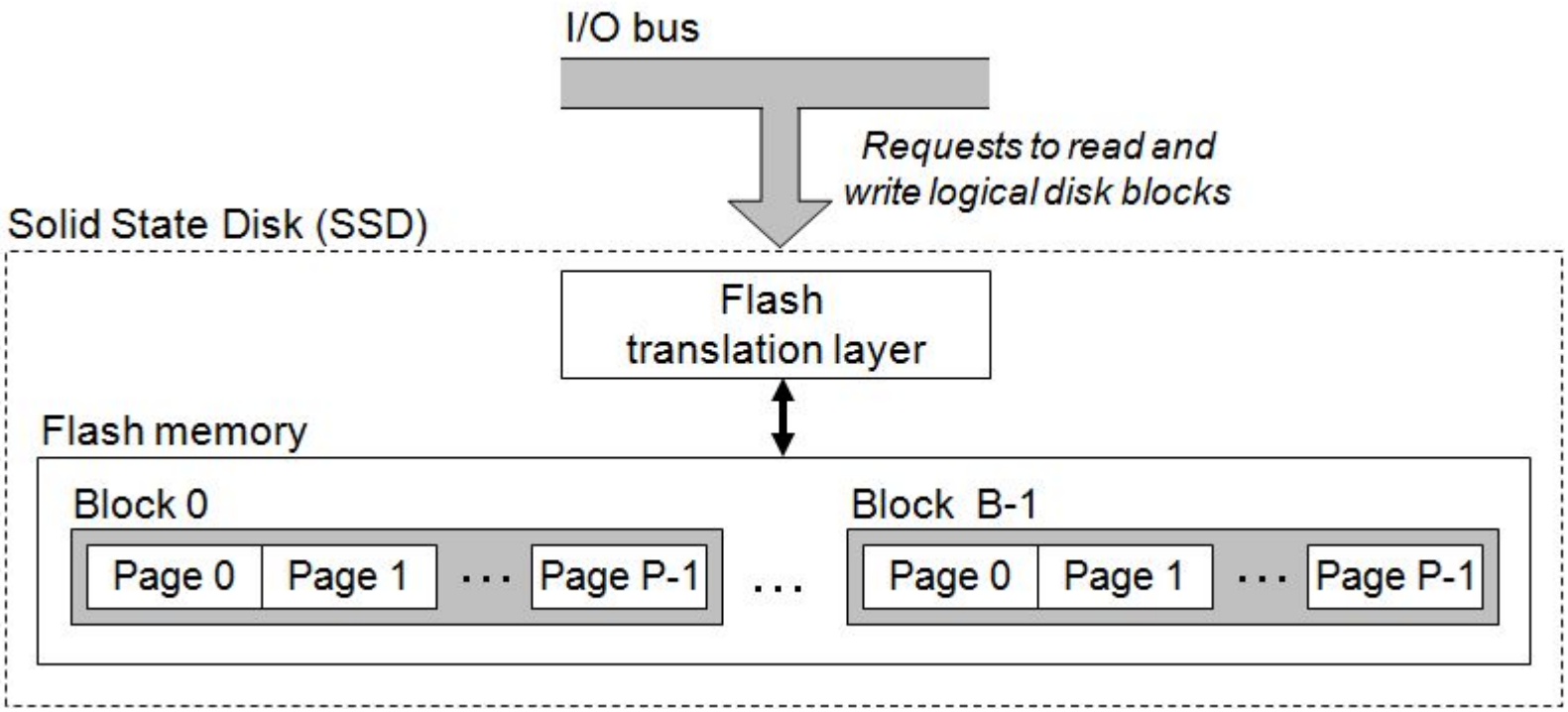
- Problem: Disks have varied and complex geometries
- Sectors/tracks may fail, today's drives automatically remap bad sectors
- Too complicated for OS to keep track
- Solution:
  - HD controller provides a logical block interface
  - Sectors are assigned logical #s 0 … b-1
  - HD controller (on disk) keeps track of mapping to (surface, track, sector)
- To perform r/w:
  - OS specifies block # instead of (surface, track, sector)
  - HD controller maps that to (surface, track, sector)
  - r/w data from/to that sector

Solid State Disks

- A solid state disk is a storage technology based on flash memory
- Flash memory is a type of nonvolatile memory based on EEPROMs
  - Provides fast and durable storage for many devices
  - Digital cameras, cell phones, music players, PDAs, laptop, desktop, and server computer systems

Solid State Disks

- Can provide an alternative to rotation disk



Solid State Disks

- A SSD package plugs into a standard disk slot in the I/O bus
  - Usually USB or SATA
  - Behaves like any other disk
- Physically, disk consists of one or more flash memory chips
  - Replaces the mechanical drive in conventional disks
  - Also, contains hardware/firmware device
    - Same role as disk controller in conventional HDs
    - Translates requests for logical blocks into accesses for the underlying physical device

SSD vs. Disk

- Sequential reads/writes are faster on the SSD
  - ~66% improvement in access times
- Logical blocks **written** in random order is much **slower** for SSD
  - Order of magnitude slower
- Logical blocks **read** in random order is much **faster** for SSD

SSD vs. Disk

- Random performance due to physical characteristics of flash memory
- A flash memory consists of B blocks
  - Each block consists of P pages
    - Typically pages are 512 B – 4 KB
    - Typical block consists of 32 – 128 pages
    - Thus, total block size range from 16KB – 512 KB
- A page can be written to only after entire block is erased
  - Once block has been erased, each page can be written once with no further erasing
- A block wears out after ~100,000 writes
  - Can no longer be used

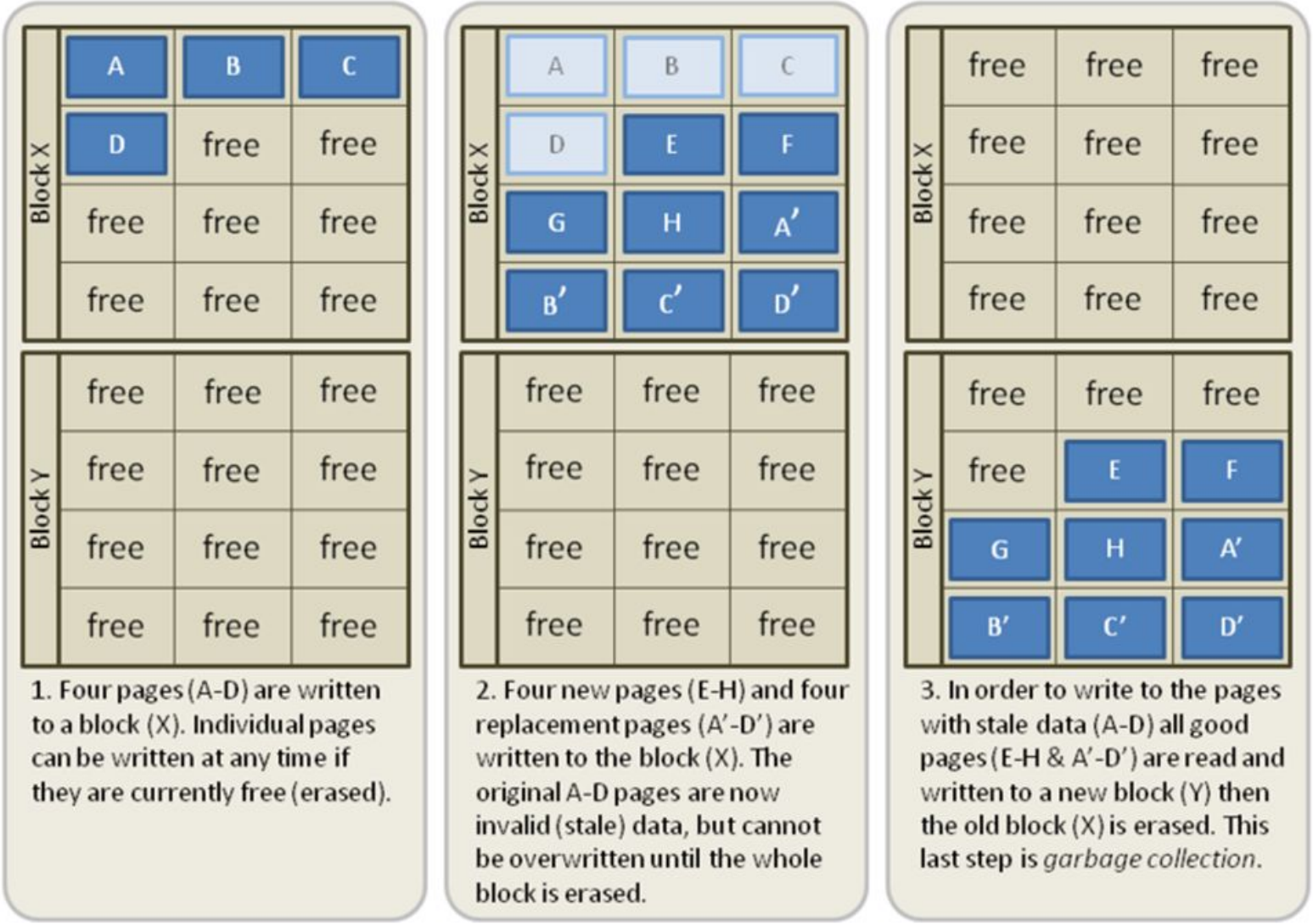
SSD vs. Disk

- Random writes are slow for two reasons
  - Erasing a block takes a relatively long time
    - ~ 1ms
  - Any pages containing existing data in block being written to must be copied to a new block before writing
- Manufactures try to solve this problem by using sophisticated logic in the flash translation layer

Block erasing

- Why can we only erase a block at a time?
  - Technically possible to erase page-by-page
  - Requires high voltage and risk of damaging nearby memory cells
  - Therefore, block erasure constriction is purely an engineering concern

Garbage Collection



1. Four pages (A-D) are written to a block (X). Individual pages can be written at any time if they are currently free (erased).

2. Four new pages (E-H) and four replacement pages (A'-D') are written to the block (X). The original A-D pages are now invalid (stale) data, but cannot be overwritten until the whole block is erased.

3. In order to write to the pages with stale data (A-D) all good pages (E-H & A'-D') are read and written to a new block (Y) then the old block (X) is erased. This last step is *garbage collection*.

SSD Advantages

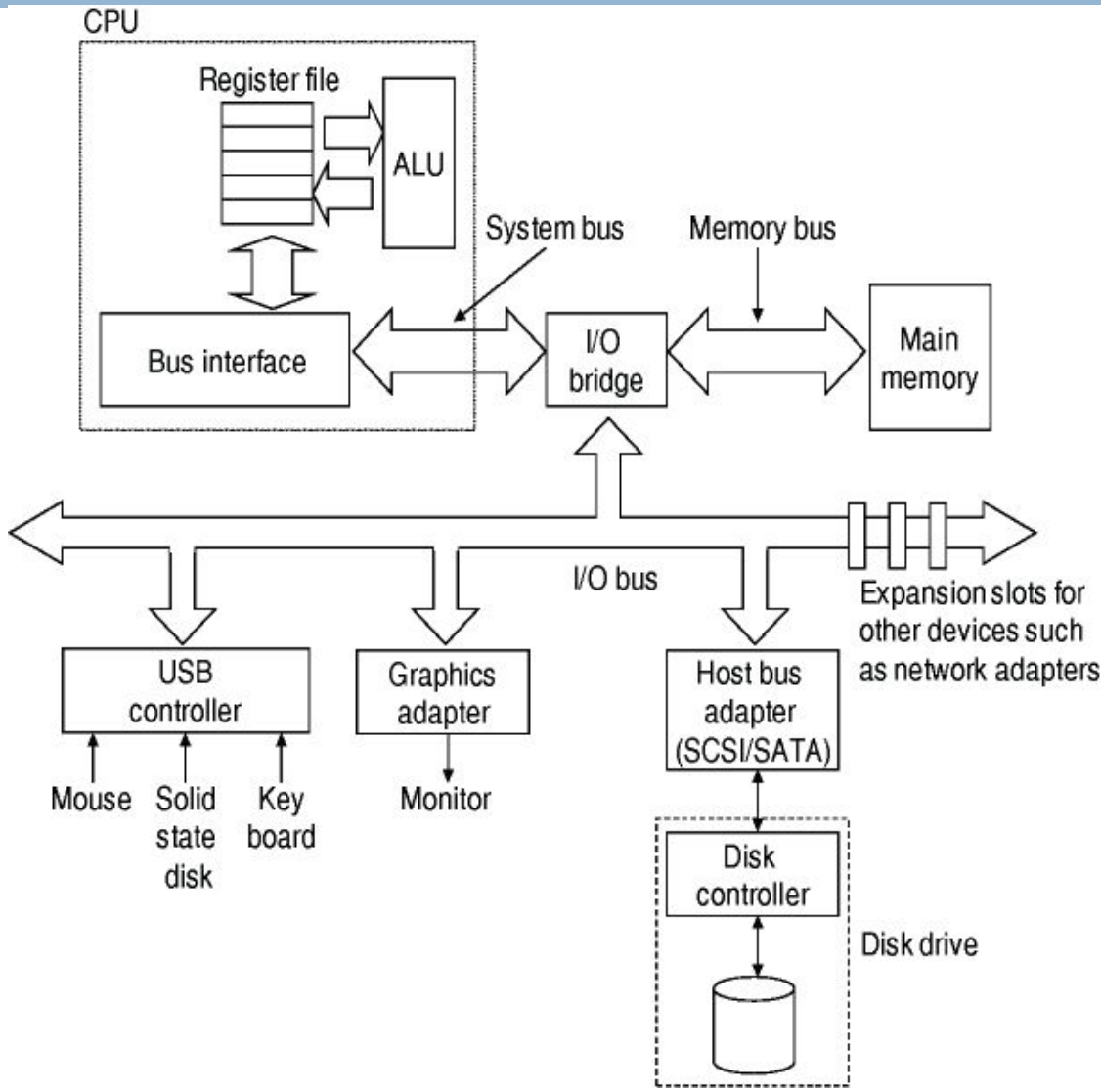
- They are built of semiconductor memory
  - No moving parts = faster random access times
  - Use less power
  - More rugged

SSD Disadvantages

- SSDs have potential of wearing out
  - Since flash blocks wear out
  - Mitigated by logic in flash translation layer
    - Attempts to spread erasures evenly across all blocks
- More expensive

Accessing Disks

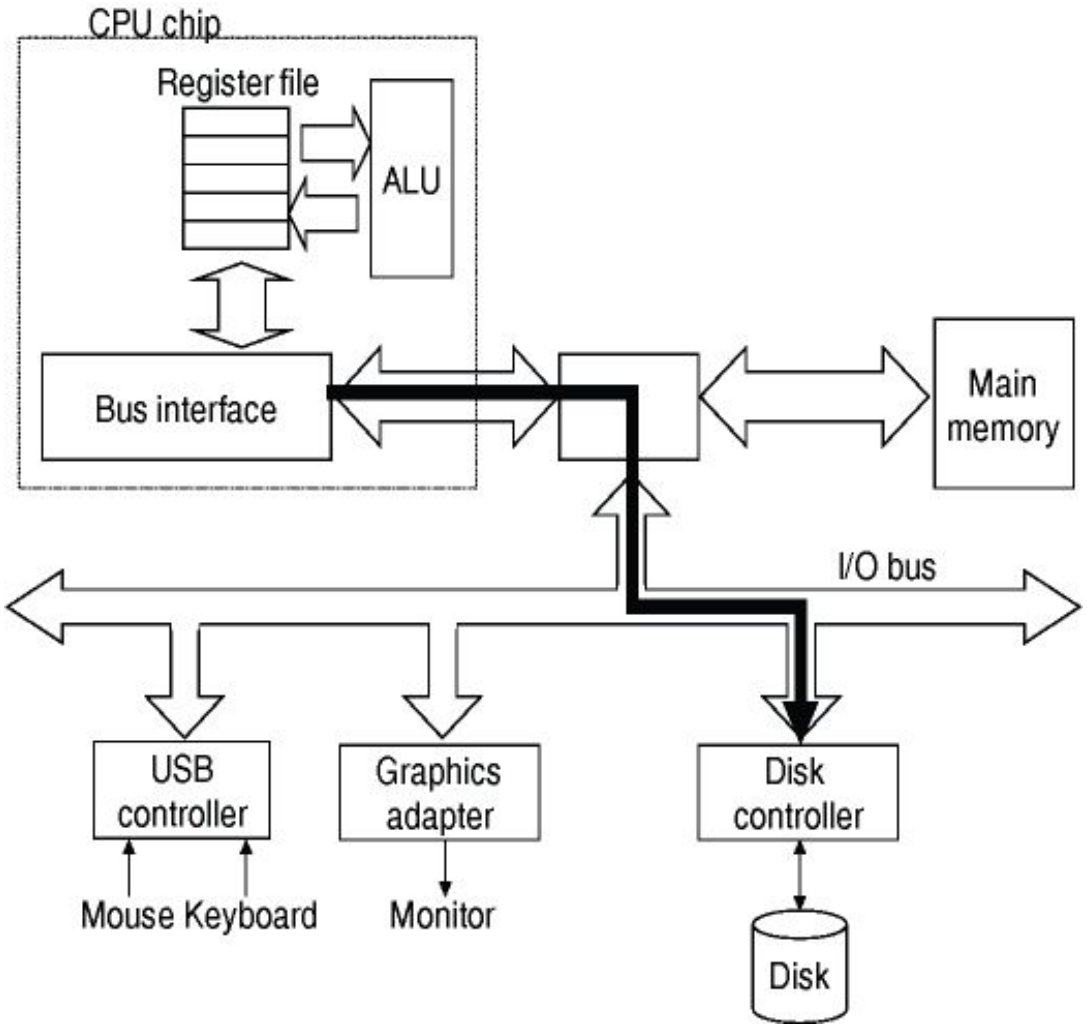
- Hardware interconnect
- I/O Bridge: seen before
- I/O Bus: links all I/O devices to CPU
- I/O Controllers: translate bus signals to device signals



Accessing Disks

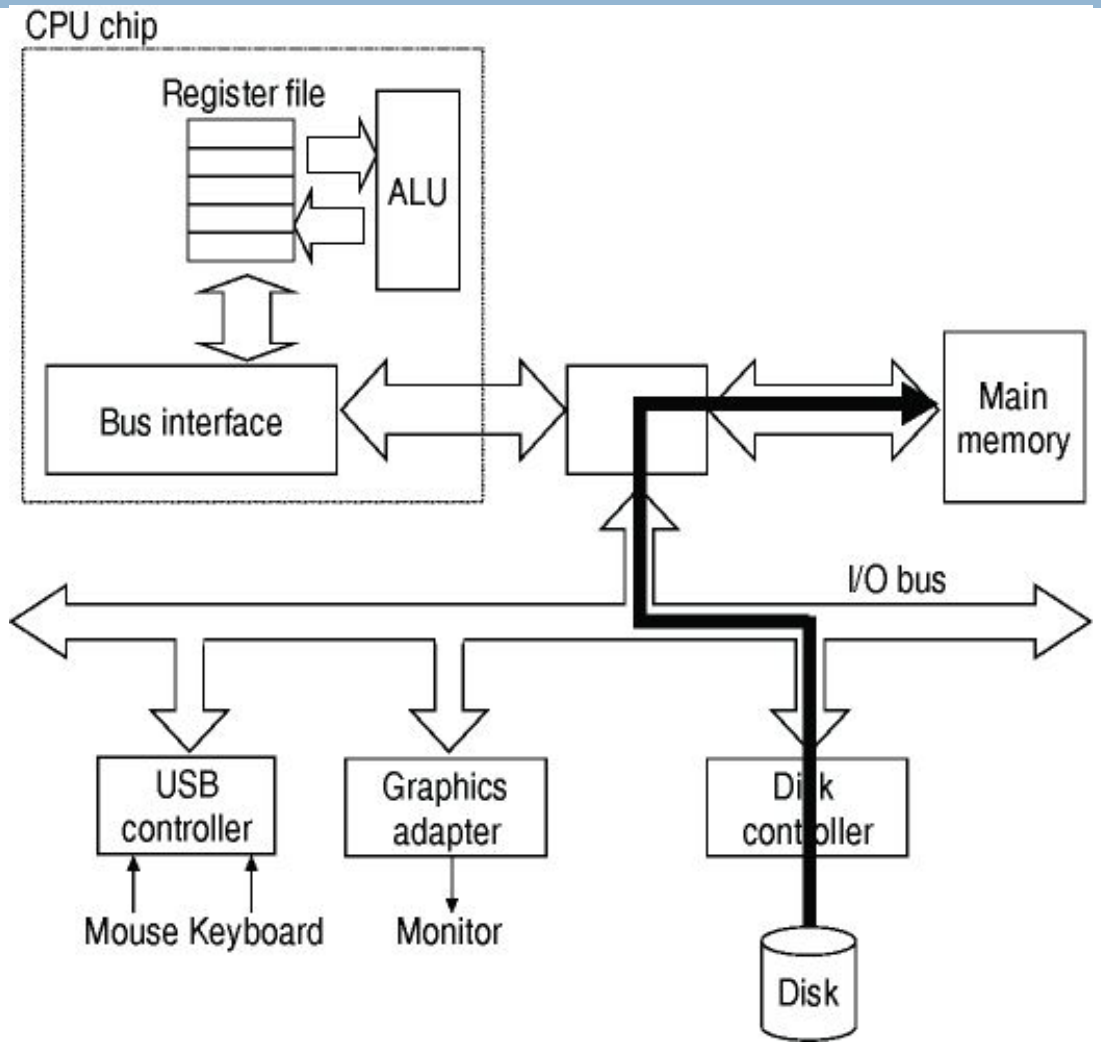
- To send/receive a word to a device, CPU writes/reads word to/from special location in memory
  - Block of addresses reserved for

  Communicating with I/O devices
  - Each address is called a port
  - Corresponds to a particular I/O device
- r/w request is sent via system bus



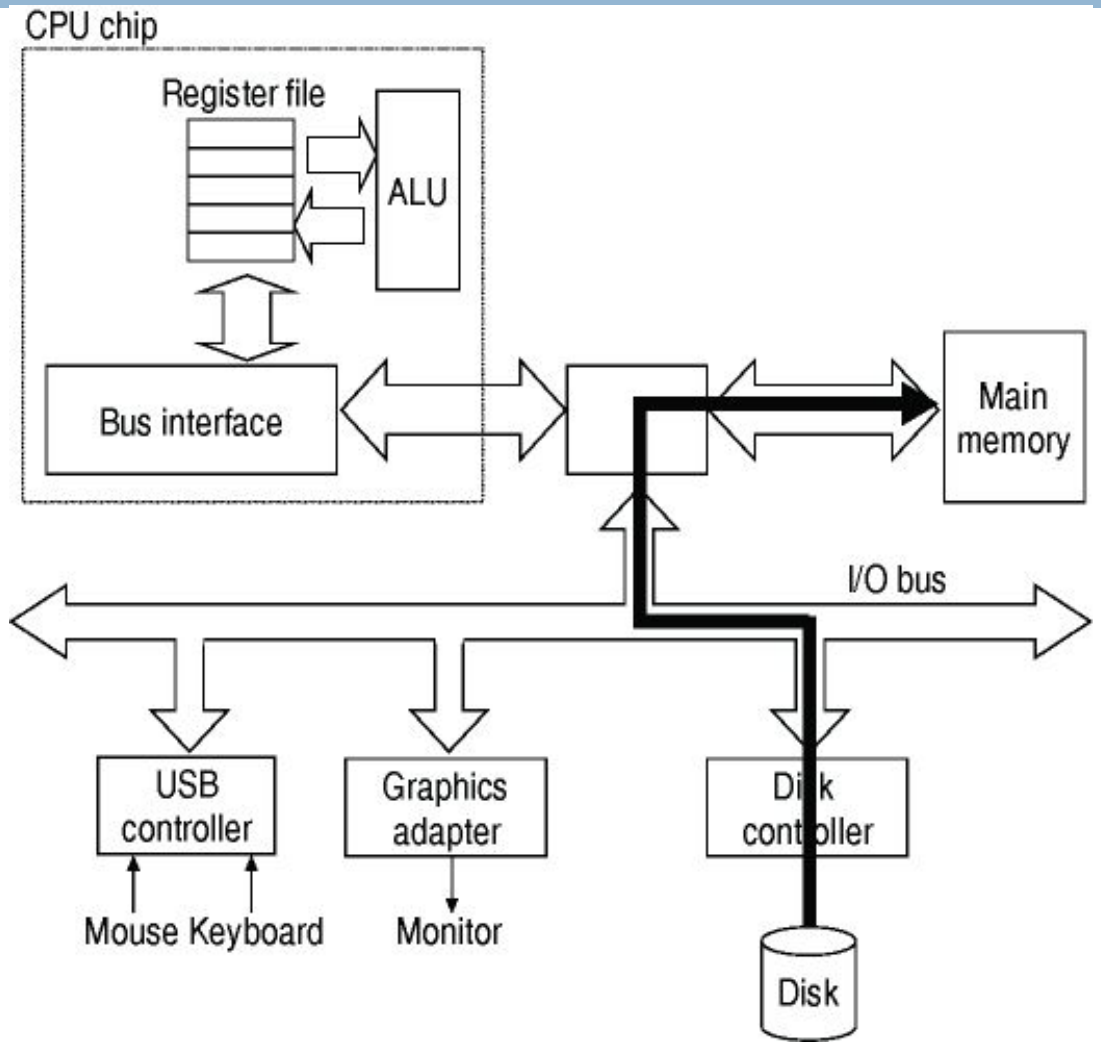
Accessing Disks

- I/O controller intercepts request
- Forwards it to corresponding device via I/O bus
- Good method for exchanging small amounts of data with device
- For large amounts of data computers use DMA



Accessing Disks

- Direct Memory Access
  - CPU sends r/w request to HD
  - Includes memory address of where to write/read the data
  - Includes size of data
  - HD controller and IO controller collaborate
  - Data is sent directly to/from memory from/to HD
  - Bypassing the CPU
  - Much more efficient!

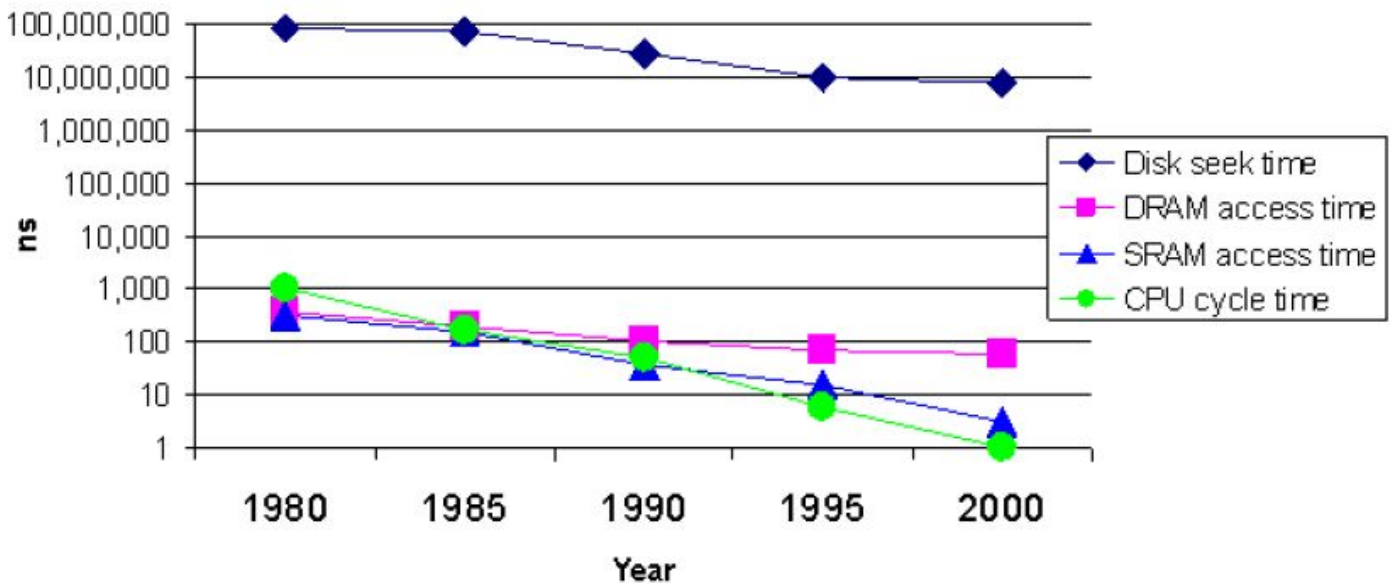


Direct Memory Access

- Example: A HD read
  - CPU uses memory mapped IO to send the following to HD controller
    - # of logical block to read
    - Memory address where to store the block
  - CPU proceeds to do something else
  - Meanwhile: HD controller converts # to (sector, cylinder, surface)
  - Reads sector from disk
  - Sends address to IO controller with instruction to DMA
  - Sends the sector over the IO and memory busses to the memory
  - Notifies CPU when the transfer is done
  - During this time CPU does other work
    - Takes 16ms to read a sector
    - A 1GHz CPU could execute 16 million instructions in this time
    - Waiting for data would be extremely wasteful

Storage Technology Trends

Device	1980	1985	1990	1995	2000	Overall
SRAM	300ns	150ns	35ns	15ns	3ns	100
DRAM	375ns	200ns	100ns	70ns	60ns	6
Disk	87ms	75ms	28ms	10ms	8ms	11
CPU	1000ns	166ns	50ns	6ns	1.6ns	600



Storage Technology Trends

- Observations
  - CPU speed is out-pacing all storage
  - SRAM is keeping up, but
  - DRAM and Disk are sorely lacking
  - SRAM is too expensive to use for main memory
  - DRAM and disk are main storage devices
  - Creates the CPU-memory gap, CPU is limited by speed of memory and disk
  - To bridge gap: modern systems use SRAM based caches
  - This approach works because most programs exhibit good locality